

Assignment 16 Report

Multithreaded Producer-Consumer System Using POSIX Threads and Semaphores

Name: ARITRA TALUKDAR

Roll Number: CSB24034

May 15, 2026

Contents

1	Objective	2
2	Introduction	2
3	Theory	2
3.1	Threads	2
3.2	Race Condition	2
3.3	Semaphore	3
3.4	Critical Section	3
3.5	Producer-Consumer Problem	3
4	Algorithm	3
4.1	Producer Algorithm	3
4.2	Consumer Algorithm	3
5	Program Code	4
6	Explanation of the Program	6
6.1	Shared Buffer	6
6.2	Semaphores Used	6
6.3	Producer Thread	6
6.4	Consumer Thread	6
7	Output	7
8	Advantages of Synchronization	7
9	Applications	7
10	Conclusion	8

1 Objective

The objective of this assignment is to develop a multithreaded C program using POSIX threads where multiple threads coordinate access to a shared resource using semaphores. The program demonstrates synchronization between producer and consumer threads while safely accessing shared memory.

The assignment helps in understanding:

- Multithreading using POSIX threads
- Shared resources and critical sections
- Race conditions
- Synchronization using semaphores
- Mutual exclusion
- Producer-consumer problem

2 Introduction

Multithreading allows multiple threads to execute concurrently within the same program. Threads share the same memory space and resources, making communication efficient. However, improper access to shared resources may lead to incorrect behavior known as race conditions.

Synchronization mechanisms such as semaphores and mutexes are used to coordinate thread execution and ensure safe access to shared resources.

In this assignment, the Producer-Consumer problem is implemented using:

- POSIX threads
- Semaphores
- Shared circular buffer

3 Theory

3.1 Threads

A thread is the smallest unit of execution inside a process. Multiple threads can run concurrently and perform different tasks simultaneously.

3.2 Race Condition

A race condition occurs when multiple threads access and modify shared data simultaneously without proper synchronization, resulting in inconsistent outputs.

3.3 Semaphore

A semaphore is a synchronization variable used to control access to shared resources.

Important semaphore operations:

- `sem_wait()` : Decreases semaphore value and waits if resource unavailable.
- `sem_post()` : Increases semaphore value and signals waiting threads.

3.4 Critical Section

The part of the program where shared resources are accessed is called the critical section. Only one thread should execute the critical section at a time.

3.5 Producer-Consumer Problem

The producer thread generates data and stores it in a shared buffer. The consumer thread removes and processes data from the buffer.

Synchronization ensures:

- Producer waits if buffer is full
- Consumer waits if buffer is empty
- Safe access to shared buffer

4 Algorithm

4.1 Producer Algorithm

1. Generate an item
2. Wait for an empty slot
3. Enter critical section
4. Insert item into buffer
5. Exit critical section
6. Signal that buffer contains a new item

4.2 Consumer Algorithm

1. Wait for available item
2. Enter critical section
3. Remove item from buffer
4. Exit critical section
5. Signal that buffer has an empty slot

5 Program Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5 #include <unistd.h>
6
7 #define BUFFER_SIZE 5
8 #define ITEMS_TO_PRODUCE 10
9
10 int buffer[BUFFER_SIZE];
11
12 int in = 0;
13 int out = 0;
14
15 sem_t empty;
16 sem_t full;
17 sem_t mutex;
18
19 void* producer(void* arg)
20 {
21     int item;
22
23     for(int i = 1; i <= ITEMS_TO_PRODUCE; i++)
24     {
25         item = i;
26
27         sem_wait(&empty);
28
29         sem_wait(&mutex);
30
31         buffer[in] = item;
32
33         printf("Producer produced item %d at position %d\n",
34             item, in);
35
36         in = (in + 1) % BUFFER_SIZE;
37
38         sem_post(&mutex);
39
40         sem_post(&full);
41
42         sleep(1);
43     }
44
45     pthread_exit(NULL);
46 }
47
48 void* consumer(void* arg)
```

```

49 {
50     int item;
51
52     for(int i = 1; i <= ITEMS_TO_PRODUCE; i++)
53     {
54         sem_wait(&full);
55
56         sem_wait(&mutex);
57
58         item = buffer[out];
59
60         printf("Consumer consumed item %d from position %d\n",
61             item, out);
62
63         out = (out + 1) % BUFFER_SIZE;
64
65         sem_post(&mutex);
66
67         sem_post(&empty);
68
69         sleep(2);
70     }
71
72     pthread_exit(NULL);
73 }
74
75 int main()
76 {
77     pthread_t producerThread;
78     pthread_t consumerThread;
79
80     sem_init(&empty, 0, BUFFER_SIZE);
81     sem_init(&full, 0, 0);
82     sem_init(&mutex, 0, 1);
83
84     pthread_create(&producerThread, NULL, producer, NULL);
85     pthread_create(&consumerThread, NULL, consumer, NULL);
86
87     pthread_join(producerThread, NULL);
88     pthread_join(consumerThread, NULL);
89
90     sem_destroy(&empty);
91     sem_destroy(&full);
92     sem_destroy(&mutex);
93
94     printf("\nExecution completed successfully.\n");
95
96     return 0;
97 }

```

6 Explanation of the Program

6.1 Shared Buffer

The shared buffer stores produced items temporarily before consumption.

```
1 int buffer[BUFFER_SIZE];
```

6.2 Semaphores Used

- `empty` : Counts available empty slots
- `full` : Counts filled slots
- `mutex` : Ensures mutual exclusion

6.3 Producer Thread

The producer:

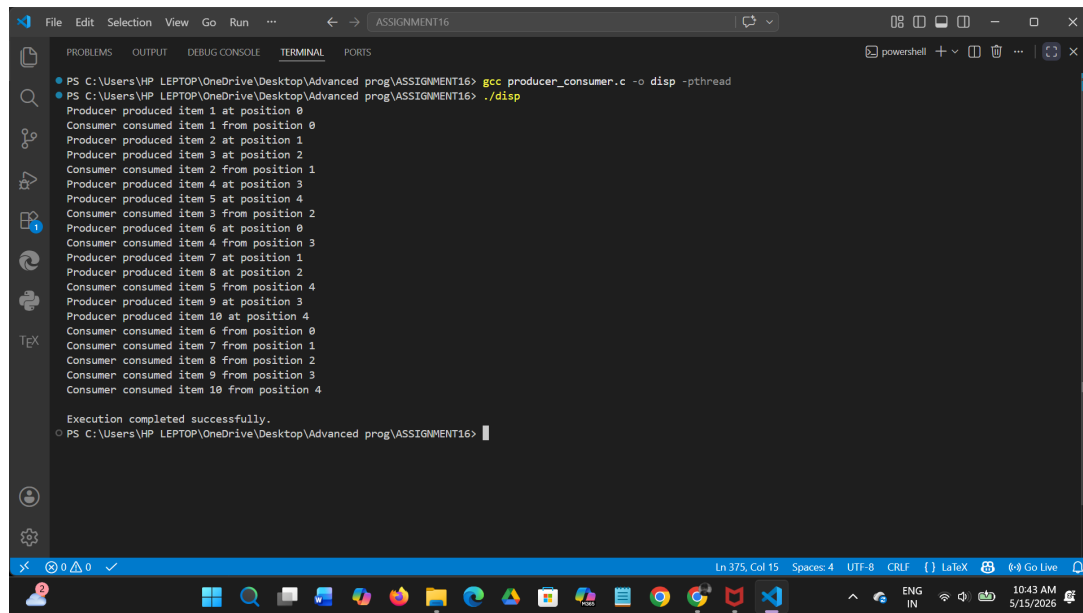
- Waits for empty space
- Locks the critical section
- Inserts item into buffer
- Unlocks critical section
- Signals consumer

6.4 Consumer Thread

The consumer:

- Waits for available item
- Locks the critical section
- Removes item from buffer
- Unlocks critical section
- Signals producer

7 Output



```
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT16> gcc producer_consumer.c -o disp -pthread
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT16> ./disp
Producer produced item 1 at position 0
Consumer consumed item 1 from position 0
Producer produced item 2 at position 1
Producer produced item 3 at position 2
Consumer consumed item 2 from position 1
Producer produced item 4 at position 3
Producer produced item 5 at position 4
Consumer consumed item 3 from position 2
Producer produced item 6 at position 0
Consumer consumed item 4 from position 3
Producer produced item 7 at position 1
Producer produced item 8 at position 2
Consumer consumed item 5 from position 4
Producer produced item 9 at position 3
Producer produced item 10 at position 4
Consumer consumed item 6 from position 0
Consumer consumed item 7 from position 1
Consumer consumed item 8 from position 2
Consumer consumed item 9 from position 3
Consumer consumed item 10 from position 4

Execution completed successfully.
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT16>
```

Figure 1: Program Output Screenshot

8 Advantages of Synchronization

- Prevents race conditions
- Ensures correct execution order
- Avoids data corruption
- Provides safe shared-memory access
- Enables proper thread communication

9 Applications

- Operating Systems
- Database Systems
- Web Servers
- Parallel Computing
- Real-Time Systems
- Process Scheduling

10 Conclusion

This assignment successfully demonstrates synchronization in multithreaded programming using POSIX threads and semaphores. The producer-consumer problem was implemented to coordinate multiple threads accessing a shared buffer safely.

Semaphores ensured:

- Proper synchronization
- Mutual exclusion
- Safe shared-memory access
- Correct thread communication

Thus, synchronization prevents inconsistent behavior and enables reliable concurrent execution.